Workshop No. 2 Systems Analysis

Authors

Leudo Mosquera Jose Luis - 20251020148

Rojas Hernandez David Santiago - 20251020153

Hernandez Ortiz Adrian Mauricio - 20251020147

Johan Sebastian Candia Marin - 20251020139

Professor

Carlos Andrés Sierra, M.Sc

Francisco José de Caldas District University

Bogotá D.C., Colombia — March 2026-1

1 Executive summary

Workshop 1 identified that the system effectively structures the connection between volunteers and educational projects, integrating key variables such as skills, availability, and location. However, some potential bottlenecks were also noted, particularly in the assignment process, which depends on the quality and timeliness of the data entered by volunteers and institutions. On the other hand, scheduling availability and fluctuations in the number of volunteers can affect the system's efficiency. When considering the expectations of tutors and students, it is observed that students require constant, accessible support tailored to their specific academic needs, while tutors seek flexibility, clear scheduling, and projects aligned with their skills. This implies that the system must ensure accurate and balanced matching, where both parties benefit. It is also key that the platform offers follow-up and feedback, as this influences both the motivation of volunteers and the academic progress of students. Finally, based on the survey graphs, patterns can be identified, such as the academic areas with the highest demand, the times with the greatest volunteer availability, and the main motivations for participating. These results reinforce the importance of a well-designed matching algorithm and proper information management.

2 Requirements analysis

ID	Requirement	Priority
01	Volunteer registration with information on skills, experience, availability, and location.	High
02	Educational institution registration with definition of academic need.	High
03	Weighted compatibility algorithm (skill, schedule, location, experience).	Critical
04	Schedule coordination without conflicts and session confirmation.	High
05	Session tracking indicators and measurement of academic impact.	Medium
06	Report generation (volunteer hours, session results, improvement rates).	Medium
07	Notification system (email) for confirmations and reminders.	High
08	Administration panel for supervision, cancellations, and system configuration.	Medium

Table 1: Functional Requirements

Category	Requirement	Target Metric
Performance	Matching algorithm response time	less time
Scalability	Concurrent user capacity	80 simultaneous users
Availability	System uptime	99% monthly uptime
Usability	Task completion for volunteer registration	Under 5 minutes, no training required
Maintainability	Module update without system-wide impact	Modular design compatible with microservices

Table 2: Non-Functional Requirements

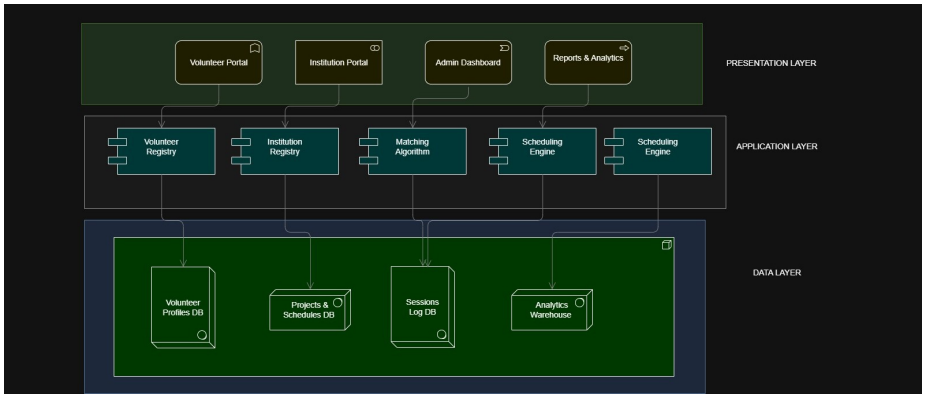
The functional requirements of the system include the registration of volunteers along with their skills (math, reading, English, technology), teaching experience,

availability, and location; the registration of institutions and their academic needs; a matching algorithm; scheduling; tracking sessions; and generating impact reports. Non-functional requirements include adaptability to basic technological infrastructure (internet and devices), flexibility in response to changes in volunteer supply and educational demand, and usability for users without advanced technical training. Baseline information was collected through a digital survey of students at the Francisco José de Caldas District University, identifying availability, skills, and preferences regarding delivery methods (in-person, virtual, or blended).

3 System architecture

The platform is structured as a three-tier architecture, ensuring a clear separation of responsibilities, independent scalability of components, and easy maintenance. This structure directly addresses the modular design requirements identified in the component analysis from Workshop 1.

3.1 Architecture diagram



3.2 Layer Description

Presentation Layer Delivers the user experience through web portals and dashboards. The Volunteer Portal manages registration, profile administration, and session tracking. The Institution Portal enables the registration and monitoring of academic projects. The Administration Dashboard provides system oversight, and the Reports and Analytics module presents visual impact indicators. Application Layer Contains all business logic. The Volunteer Registration and Institution Registration modules manage data ingestion and validation. The Matching Algorithm is the system's central engine: it calculates weighted compatibility scores across four dimensions—skill alignment, schedule overlap, geographic proximity, and teaching experience. The Scheduling Engine coordinates

confirmed assignments while avoiding conflicts. The Impact Monitor records session completions and aggregates academic improvement metrics. Data Layer: The entire system state is persisted in four dedicated databases: Volunteer Profile Database, Projects and Schedules Database, Session Log Database, and an Analytics Repository for report queries. Because the scope of the project is small, we plan to implement Google Sheets as a temporary database using the Google Sheets API, which allows reading and writing data from Python.

3.3 Design of the Potential Matching Algorithm to Be Implemented

The matching algorithm is the core intelligence of the system. It calculates a composite compatibility score C(v, p) for each volunteer-project pair using a weighted formula:

Community Education Volunteer Coordination Platform — Workshop 2

$$C(v, p) = w1 \cdot Skill(v,p) + w2 \cdot Schedule(v,p) + w3 \cdot Location(v,p) + w4 \cdot Experience(v,p)$$

Component	Description	Default Weight	Data Source
Skill(v,p)	Overlap between volunteer subject competencies and project academic needs	**w1 = 0.40**	Survey P2 + Project registration
Schedule(v,p)	Percentage of required time slots covered by the volunteer's availability windows	**w2 = 0.30**	Survey P5, P6
Location(v,p)	Geographic proximity score (in-person) or connectivity quality score (virtual)	**w3 = 0.15**	Survey P7 + Institution address
Experience(v,p)	Pedagogical experience score mapped to the student grade level requirements	**w4 = 0.15**	Survey P4

P refers to the questions (Questions) in the survey form they designed and administered in Workshop 1. Specifically, those in the data collection section (section 2.7 of the document):

Survey Reference Table — Matching Algorithm		
Community Education Volunteer Coordination Platform — Workshop 2		
Code	**Original Survey Question**	**What it contributes to the algorithm**
P1	What is your field of study or profession?	General context for the volunteer profile. Not directly used in C(v,p) but informs profile categorisation.
P2	In which subjects or academic areas do you consider yourself competent to teach?	Directly feeds Skill(v,p) — the highest-weight component (w1 = 0.40). Maps volunteer competencies to project subject needs.
P3	What would motivate you to participate in an educational volunteering program?	Does not enter the algorithm. Used for communication design and volunteer retention strategies.
P4	Have you previously carried out tutoring, mentoring or peer support activities?	Feeds Experience(v,p) — distinguishes between formal, informal, or no prior experience (w4 = 0.15).
P5	How many hours per week would you be willing to dedicate to tutoring or mentoring activities?	Feeds Schedule(v,p) — defines the volunteer's maximum weekly workload cap (w2 = 0.30).
P6	What days or time slots are you available?	Complements Schedule(v,p) — enables direct cross-referencing of availability windows against project timetables (w2 = 0.30).
P7	Would you prefer to provide the support in-person, virtually, or both?	Feeds Location(v,p) — determines whether geographic proximity or connectivity quality is applied (w3 = 0.15).

4 Implementation strategy

4.1 Implementation Phases

- Phase 1: Foundations or pillars of the system
  - Set up the project repository: The folder on GitHub where all the team's code will be stored. You might also want to consider a CI/CD pipeline, where every time someone uploads new code to GitHub, the system automatically tests and deploys it without anyone having to do it manually. CI (Continuous Integration) = automatically checks that the new code doesn't break what was already working.
  - CD (Continuous Deployment) = if everything is okay, it automatically deploys it to the server
  - Implement the volunteer and institution registration modules with basic profile management.
  - Deploy the Google Sheets schema as a temporary database using the Google Sheets API
- Phase 2: Matchmaking Engine
  - Develop and perform unit tests on the compatibility scoring algorithm.
  - Integrate it with the volunteer and project databases.

  - Expose the matching endpoint with ranked results. The system can recommend volunteers sorted by compatibility based on the compatibility scoring algorithm.

- Phase 3: Scheduling and Notifications
  - Build a conflict-free scheduling engine.
  - Implement the session confirmation workflow.
  - An added benefit of the system would be to integrate an email and SMS notification service for assignments, reminders, and cancellations.

- Phase 4: Impact Monitoring and Reporting
  - Develop session registration forms for volunteers and organizations.
  - Create impact metrics (attendance rate, % academic improvement, volunteer hours), while also taking into account feedback from users of the volunteer system.
  - Generate reports

5 Risk Management and Complexity

Workshop 1 identified key sensitivity factors and emerging behaviors within the system. This section outlines design mechanisms for managing them.

Sensitivity Factor	Ref. Workshop 1	Mitigation Mechanism
Variability in volunteer supply	Section 3.2	Waiting queue and temporary matching assignments
Scheduling conflicts	Section 3.2	Conflict detection before confirmation
Low connectivity in communities	Section 2.5	App with session recording in offline mode
Changes in student demand	Section 3.4	Monthly or weekly reports during demand peaks

6 Performance and Quality Assurance

6.1 Performance metrics

Metric	Target
Volunteer–Student matching response time	< 1 minute
System availability (Monthly)	> 90%
Assignment accuracy (Volunteer and student satisfaction)	> 80% positive feedback
Reduction in school dropout	Downward trend

6.2 Quality Assurance Strategy

- Unit tests for all algorithm functions.
- Integration tests across different scenarios.
- Load testing, simulations targeting 100 or more concurrent users without performance degradation.
- Usability testing, pilot sessions with university students from the UD.
- Continuous monitoring, automatic alerts for error rates and long response times.

7 conclusion

It is concluded that the proposed system has evolved into a much clearer and more organized structure, moving from a conceptual idea to a more technical and practical design. The definition of functional and non-functional requirements made it possible to more precisely define the system's scope, while the three-tier architecture facilitates understanding of how the different components interact. Furthermore, the design of the matching algorithm is established as the core of the system, as it enables the transformation of collected information into efficient assignments, which directly addresses the issues identified in the previous workshop.

On the other hand, the implementation strategy and risk management demonstrate that the system is not only designed with its ideal operation in mind, but also with the potential problems that may arise in a real-world environment. Issues such as volunteer variability, scheduling conflicts, or connectivity were addressed through specific mitigation mechanisms, which strengthens the project's viability. Likewise, the definition of performance metrics and quality strategies demonstrates the intention to build a measurable, evaluable, and continuously

improving system, ensuring it meets the objectives of educational impact and operational efficiency.

As next steps, it is considered essential to move forward with the implementation of the functional prototype, starting with the basic registration modules and subsequently integrating the matching algorithm to conduct pilot tests that validate the system's performance in a real-world environment.